Investigating the Robustness of Audio Features Under Concept Drift in Android Malware Detection

1st Mussa Phiri
Department of Computer Science
University of Zululand
KwaDlangezwa, South Africa
202073198@stu.unizulu.ac.za

2nd Paul Tarwireyi
Department of Computer Science
University of Zululand
KwaDlangezwa, South Africa
tarwireyip@unizulu.ac.za

3rd Alfredo Terzoli
Department of Computer Science
University of Zululand
KwaDlangezwa, South Africa
terzolia@unizulu.ac.za

***Abstract*—Android malware detection is a continuous challenge, as attackers frequently change their techniques and the Android ecosystem evolves. Machine learning models trained on historical data often degrade in performance due to concept drift, where the statistical properties of features change over time. Although most research has examined permissions and API calls, little is known about the stability of audio-based features. In this study, we investigate the temporal stability of audio features in Android applications. Using three drift measures, Population Stability Index (PSI), Jensen–Shannon distance (JS), and Kolmogorov-Smirnov statistic (KS), we evaluate feature families in quarterly malware bins. Results show that drift is widespread but uneven: features such as Zero-Crossing Rate (ZCR) and Chroma exhibit high instability, while spectral descriptors such as Spectral Flatness remain stable. A drift gap analysis confirms that the instability increases with the temporal distance. Finally, negative Spearman correlations between drift metrics and weighted F1-scores of non-incrementally trained models show that drift contributes directly to performance degradation. These findings establish that audio features are affected by concept drift and provide a baseline for future work on adaptive and multimodal detection strategies.**

***Index Terms*—Concept Drift, Android Malware Detection, Population Stability Index, Jensen–Shannon distance, Kolmogorov-Smirnov Test, Spearman Correlation.**

I. Introduction

Smartphones have become indispensable in daily life, with Android remaining the dominant operating system worldwide. This is primarily due to its open-source nature and high customizability. The operating system currently accounts for over 80% of the global mobile market [1]. However, this popularity makes Android a prime target for cybercriminals. According to [1], Android accounts for most mobile malware attacks, registering 98% of mobile attacks. Attackers continuously evolve their tactics to evade detection, and to address this, machine learning–based malware detection has gained traction, often relying on static and dynamic features such as API calls, permissions, and bytecode instructions [2] [3]. Recently, alternative feature representations have emerged, including images derived from binaries [3] and audio-based transformations of Android Package Files (APK) [4]. The conversion of APK files to audio allows for the extraction of audio features, including chroma, spectral, and cepstral features. Beyond offering a fresh perspective, this approach is effective in handling obfuscated or encrypted binaries, as it can capture underlying patterns or anomalies that might be missed when using binary format [5]. Furthermore, the lightweight nature of audio processing makes it suitable for large-scale analysis and near-real-time applications.

Although these approaches have demonstrated promising classification performance, their long-term reliability has not yet been studied. This study seeks to investigate the robustness of Android audio-based features to concept drift. Concept drift refers to changes in the statistical properties of data over time, which was shown to lead to performance degradation of machine learning models [6] in textual-based features. In the cybersecurity field, drift is especially prevalent as malware constantly evolves due to the continuous adaptation of tactics, techniques, and obfuscation strategies. Android, in particular, has an ecosystem that constantly evolves through regular OS updates and shifting development practices, while adversaries adapt malware strategies to evade detection.

Drift can be formally described using probability distributions over the model inputs and outputs. We denote the input features as X , Y as the true labels, and y the predictions of the model. *Concept drift* occurs when the conditional distribution $p(Y \mid X)$ changes over time, reflecting a change in the relationship between features and ground-truth labels. Other forms of drift include *prediction drift*, which is the change in $p(y \mid X)$, and *label drift*, which is the change in the distribution $p(Y)$. *Feature drift* refers to the change in the distribution of input data $p(X)$ [7].

In order to differentiate the effects of concept drift, two cases are considered. *Virtual drift* arises when $p(X)$ changes, but $p(Y \mid X)$ remains stable, which means that the feature distribution changes without impacting the predictive accuracy of machine learning models. In contrast, *real drift* occurs when $p(Y \mid X)$ itself changes, directly leading to degraded model performance. These distinctions are well established for traditional Android malware features, such as API calls and permissions. However, it is not yet known whether audio-based features are equally vulnerable to such drifts or whether certain families exhibit inherent resilience. Figure 1 shows an illustration of virtual drift versus real drift.

This paper addresses that gap by performing a statistical investigation of concept drift in audio-based features for Android malware detection. Using established drift metrics, including

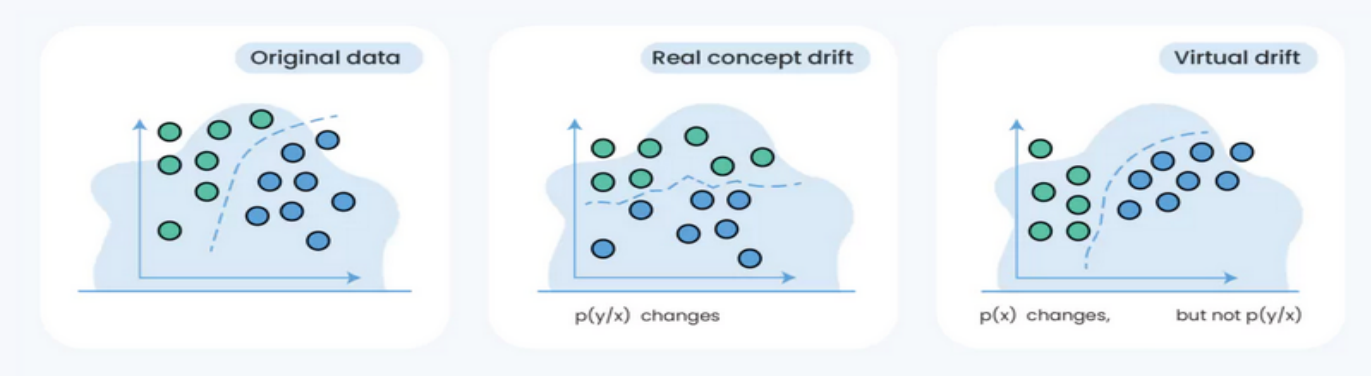


Fig. 1. Virtual Drift vs Real Drift. Source: [7]

the Population Stability Index (PSI), Kolmogorov–Smirnov (KS) test, and Jensen–Shannon (JS) distance, we measure distributional changes across a time-binned malware dataset. We further provide feature family stability rankings and correlations between drift severity and model degradation in non-incremental settings.

II. Related Work

Concept drift is a critical challenge in machine learning, arising when models trained on historical data degrade as underlying patterns evolve in new data. This section reviews prior work in two parts: general advances in drift detection and studies focused on Android malware.

A. Concept Drift in Machine Learning

The authors in [8] reviewed concept drift detection strategies across machine learning domains. They noted the common use of statistical tests (PSI, KL, JS, KS) and error-rate based detectors (DDM, EDDM, ADWIN). Ensemble and neural approaches were seen as effective but costly, while statistical and window-based methods were faster for streaming data. The study concluded that no single method dominates and highlighted gaps in non-tabular domains, such as image and audio.

The authors [9] developed CDSeer, a semi-supervised, model-agnostic drift detection framework designed for real-world deployment. Unlike prior methods such as ECHO, which required extensive manual labelling, CDSeer reduced labelling to less than 1% of data through clustering-based sampling and pseudo-labelling. It was compatible with pre-trained models and adaptable across architectures. Evaluation on eight datasets showed CDSeer achieved higher precision and recall than ECHO while matching the performance of fully supervised detectors, providing a practical solution for managing drift in deployed ML systems.

The work by [10] introduced a model-based explanation framework for concept drift using explainable AI (XAI) methods. Instead of only detecting or quantifying drift, their approach localized and segmented drift regions and then applied explanation techniques such as global feature importance, discriminative dimensionality reduction, local explanations (LIME, saliency maps), and counterfactuals. Experiments on synthetic datasets, Bayesian networks, sensor data, and images (MNIST, Fashion-MNIST, ImageNet) demonstrated the ability to highlight drifting features and visualize how drift manifests. The study underscored the importance of interpretable explanations to increase trust and improve decision-making in non-stationary environments.

B. Concept Drift in Android Malware Detection

While general machine learning research has established a wide range of techniques for detecting and managing concept drift, domain-specific studies are essential to understand how these methods perform under adversarial and evolving environments. Android malware detection is a particularly challenging domain, as attackers frequently adapt their strategies, and the Android ecosystem itself evolves rapidly. Recent works have begun to apply drift-aware methods to sustain classifier performance in this setting, primarily focusing on static features such as API calls and permissions.

[11] proposed a data stream learning pipeline that adapts both the classifier and feature extractor in response to drift, using detectors such as ADWIN and KSWIN to trigger retraining. Unlike methods that update only classifiers, refreshing feature extractors reduced feature drift and improved detection. On DREBIN and AndroZoo, ADWIN with retraining achieved F1-scores of 84.44% and 75.05%, outperforming standard approaches and stabilizing performance over time.

[12] performed a comprehensive drift analysis across 12 Android malware detectors based on string, image, and graph features. To quantify distributional change, they applied the Kolmogorov–Smirnov (K–S) test, which revealed significant shifts in feature distributions, particularly in function call graph structures under obfuscation. Their experiments showed that image-based detectors degraded the fastest, losing up to 38% F1-score, while graph-based models were more resilient but weak against unseen families. A key finding was that models trained on recent samples performed poorly on older data, underscoring the "forgetting effect" and reinforcing the need for adaptive or drift-aware methods.

[13] extended the Transcend framework into TRANSCENDENT, a rejection-based strategy for handling concept drift

in malware detection. The study used Kullback–Leibler (KL) divergence to show that prior evaluations exaggerated drift (KL = 696.66) by merging datasets, while their temporal AndroZoo dataset reflected more realistic evolution (average KL = 189.55). For drift detection, TRANSCENDENT relied on conformal evaluation with p-values, rejecting uncertain samples to prevent misclassification. Using data from 2014–2019, the approach maintained F1-scores above 0.8 for nearly two years, doubling classifier lifespan while keeping rejected samples almost entirely those that would have been errors.

[14] studied Android permissions for drift-resilient detection using the KronoDroid dataset (2008–2020, 79K apps). A dynamic Random Forest ensemble with time-based updates sustained an average F1 of 0.93 over seven years. The Kolmogorov–Smirnov test showed no significant difference between 60 and 122 permissions, confirming the smaller set's stability. Performance dips in 2015–2016 were linked to evolving ransomware families, showing that drift-aware permission models can remain effective long-term.

C. Audio-Based Android Malware Detection

Besides permissions and API calls, recent studies have explored audio-based representations of APK files for Android malware detection. These approaches treat bytecode as a signal and extract audio features, including spectral, cepstral, and chroma features.

The authors of [15] proposed an Android malware detection method using sonified audio features. Android APK files were converted to audio signals, from which 191 static audio-based features were extracted. The study utilised nature-inspired wrapper-based metaheuristic algorithms, including the Genetic Algorithm, to optimise feature selection. The best results were achieved with the Genetic Algorithm, which reduced the feature set by 50.26% and achieved a classification accuracy of 99.72% using the Light Gradient Boosting Machine (LGBM) model.

The authors in [16] proposed an Android malware detection method utilizing audio features. The authors converted .dex files from the Android package to audio files. They extracted features including chromagrams, spectral contrast, and Root Mean Square. They also utilized feature selection methods like information gain and gain ratio, which helped reduce computational costs. The authors achieved the best accuracy score of 96.6% with the KNN and ReliefF feature selection method.

The authors in [4] proposed an Android malware detection method using audio feature fusion. Android Application Package (APK) components were converted to binary and then to audio. Bark frequency Cepstral Coefficients (BFCC) features, and Gammatone Frequency Cepstral Coefficients (GFCC) features were then extracted from the CICMalAnal2017 and CICMalDroid2020 datasets. The study achieved the best classification accuracy of 98.96% using the Light Gradient Boost Machine (LGBM) model.

While these studies demonstrate the effectiveness of audio features for malware detection, they assume a stationary feature space and do not assess robustness under temporal or distributional drift. Evaluating this limitation is crucial for establishing the long-term reliability of audio-based techniques.

III. Research Method

The primary objective of this study is to evaluate the robustness of audio features to concept drift in Android malware detection. The methodology consists of three main stages: (i) transforming Android applications into audio signals and extracting feature families, (ii) quantifying temporal drift across quarterly dataset bins using multiple statistical measures, and (iii) validating the impact of drift by correlating drift magnitudes with classification performance in a non-incremental setting. Figure 2 illustrates the overall research methodology followed in this study.

A. Dataset Description

The study uses the AndroOBFS dataset [17], which contains 16,279 unique real-world obfuscated Android malware samples. The dataset only contains malware samples which are categorised into six obfuscation categories: (i) *Trivial*, (ii) *Renaming*, (iii) *Encryption*, (iv) *Reflection*, (v) *Code*, and (vi) *Mix* (a combination of two or more obfuscation methods). The dataset spans the period 2018–2020 and is partitioned into quarterly bins, allowing for temporal analysis experiments. Table I shows the quarterly distribution of the dataset.

TABLE I
AndroOBFS Distribution per Quarter

Quarter	Code	Encryption	Mix	Reflection	Renaming	Trivial
2018_Q1	613	301	80	66	140	46
2018_Q2	708	330	70	64	181	51
2018_Q3	788	390	81	88	175	53
2018_Q4	1158	548	143	123	236	81
2019_Q1	860	407	138	84	194	59
2019_Q2	707	320	109	67	163	47
2019_Q3	1160	525	205	118	270	80
2019_Q4	1341	651	250	137	328	93
2020_Q1	69	48	7	7	29	10
2020_Q2	368	178	60	43	100	29
2020_Q3	133	76	9	17	35	10
2020_Q4	86	59	13	12	32	11

B. Feature Extraction

The Android Application Package (APK) comprises multiple files required for the execution of the Android application, including classes.dex, the manifest, and the resource files. These files differ across applications, making each APK unique. However, the raw package files cannot be directly used in machine learning models. These files were first transformed into binary and then concatenated into a one-dimensional binary array, which was converted into a waveform representation (WAV). From the resulting audio signals, a diverse set of audio descriptors was extracted, including cepstral (MFCCs), spectral (centroid, bandwidth, contrast, flatness, roll-off), temporal (ZCR, RMS), and chroma (STFT, CENS, CQT) features, as well as Mel spectrograms and polynomial coefficients.

- **Mel-frequency cepstral coefficients** – These are audio features based on human perception of loudness and

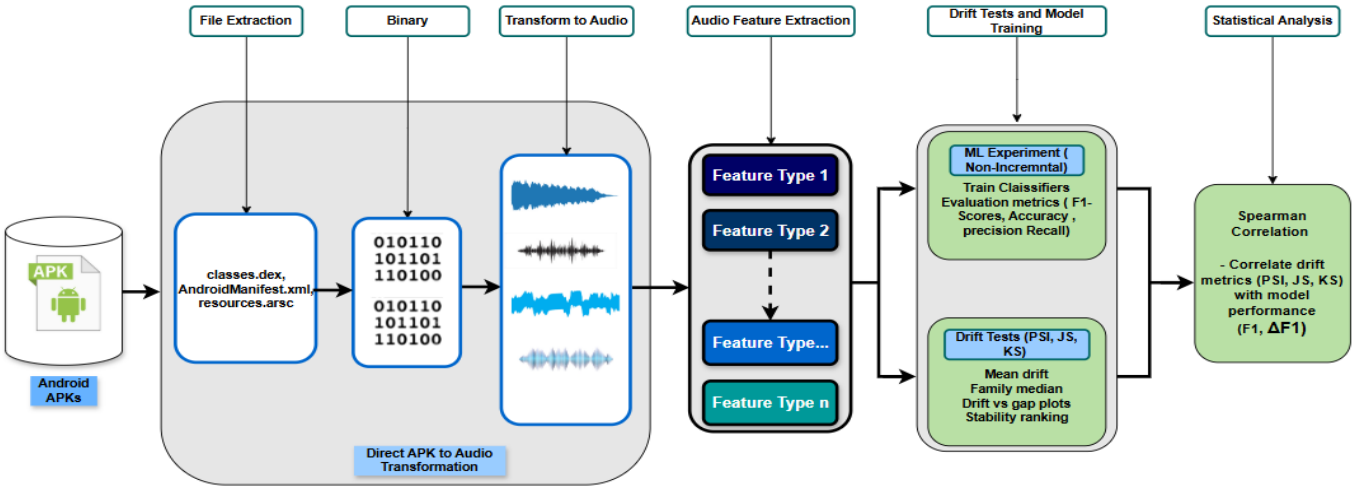


Fig. 2. Methodology. Adapted from [15]

frequency of sound using the Mel scale. The implementation process involves windowing the audio signal, applying the Discrete Fourier Transform (DFT), taking the logarithm of the signal size, warping the frequencies on a Mel scale, and applying the inverse Discrete Cosine Transform (DCT) [4].

$$f_{\text{mel}} = 2595 \cdot \log_{10}\left(1 + \frac{f}{700}\right), \tag{1}$$

where f_{mel} is the frequency on the Mel scale, and f is the frequency in Hz.

$$f = 700 \cdot \left(10^{\frac{f_{\text{mel}}}{2595}} - 1\right), \tag{2}$$

- **Zero Crossing Rate** – Measures how often the signal changes sign or crosses zero per unit time. It is a basic feature often used in speech and music classification [4].
- **Spectral Centroid** –This describes where the centre of the frequency content is located in a sound. It is found by calculating the weighted average of all the frequencies in the signal. It is linked to the perceived brightness of the sound [15]. The equation is shown below:

$$C_i = \frac{\sum_{k=1}^{W_f L} k \cdot X_i(k)}{\sum_{k=1}^{W_f L} X_i(k)} \tag{3}$$

- **Spectral Contrast** – Captures the decibel difference between peaks and troughs in the frequency spectrum, distinguishing harmonic content. High contrast usually means that the sound is clear, while low contrast may suggest that it is noisier. [16].
- **Spectral Bandwidth** – Measures the width of the frequency spectrum by quantifying the spread of frequencies around the spectral centroid. A wider bandwidth indicates more high-frequency content, while a narrower one suggests a more tonal or low-frequency-dominated signal [16].
- **Spectral Flatness** – Quantifies how noise-like or tone-like a signal is by comparing the geometric mean to the arithmetic mean of the power spectrum. A value close to 1 indicates a flat, noise-like spectrum (white noise), while values near 0 correspond to tonal signals with distinct peaks [16].
- **Spectral Rolloff** –This is the frequency below which a certain percentage, usually 85% or 95% of the total spectral energy is contained [18].

$$\sum_{k=1}^{m} X_i(k) = C \cdot \sum_{k=1}^{W_f L} X_i(k) \tag{4}$$

where C is the adopted percentage (user-defined parameter). The spectral rolloff frequency is usually normalized by dividing it by $W_f L$, so that its value lies between 0 and 1.

- **Root Mean Square (RMS)** – RMS (Root Mean Square) is used to estimate how powerful or loud a sound is within a short time window. It calculates the average energy of the audio signal and gives an indication of its overall intensity [16].
- **Chroma STFT** – Derived from the Short-Time Fourier Transform (STFT), this feature projects the audio spectrum into twelve semitone bins (one for each pitch class of the octave). It captures harmonic and melodic content by tracking how pitch class energy evolves over time [15] [19].
- **Chroma CENS** – Chroma Energy Normalized Statistics (CENS) are computed by applying additional energy normalization and statistical smoothing to the chromagram. This makes the feature more robust to dynamics and noise while retaining pitch-class distribution [20].

- **Chroma CQT** – Computed using the Constant-Q Transform (CQT), this chroma variant provides higher frequency resolution at lower frequencies and lower resolution at higher frequencies. It preserves harmonic structure by mapping energy into twelve pitch classes [20].
- **Mel-Spectrogram** – A time–frequency representation where the frequency axis is mapped to the perceptually motivated mel scale. Unlike MFCCs, which compress spectral shape into cepstral coefficients, the mel-spectrogram preserves detailed temporal and spectral patterns, making it useful for distinguishing subtle audio characteristics [16] [20].
- **Poly Features** – These features fit an n-th order polynomial to the power spectrum of each frame, returning the polynomial coefficients. They capture the overall spectral trend and curvature, serving as a compact descriptor of spectral shape [16].

C. Statistical Drift Tests

To capture distributional changes, drift was computed between consecutive quarterly bins of the AndroOBFS dataset (2018–2020).In order to quantify temporal instability in audio features, three statistical drift measures were used, including the Population Stability Index (PSI), the Kolmogorov–Smirnov (KS) test, and the Jensen–Shannon divergence (JS). Drift was first computed at the *coefficient level* for each feature family. For multi-dimensional features such as MFCC and Chroma, every coefficient was expanded and evaluated individually across consecutive quarterly bins. Family-level statistics were then obtained by aggregating these coefficient scores, reporting both the median and mean values per bin-pair. This two-level approach allowed for both fine-grained analysis of coefficient behaviour and broader comparisons across entire feature families.

1) Population Stability Index (PSI): The Population Stability Index (PSI) measures the difference between a given distribution and a reference distribution, such as datasets from different quarters or years. A PSI below 0.1 indicates stability, 0.1 to 0.25 suggests moderate change, and values above 0.25 signify significant drift [21].

$$\text{PSI} = \sum_i (p_i - q_i) \times \ln\left(\frac{p_i}{q_i}\right) \tag{5}$$

where p_i and q_i are the proportions of the i-th bin in the new and baseline distributions, respectively.

2) Kolmogorov–Smirnov (KS) Test: The Kolmogorov–Smirnov test is a non-parametric method used to assess whether two distributions differ significantly. It calculates the maximum distance between the empirical cumulative distribution functions (ECDFs) of two samples [22]:

$$D = \sup_x |F_n(x) - G_m(x)| \tag{6}$$

where $F_n(x)$ and $G_m(x)$ are the ECDFs of the two samples, and $\sup$ is the supremum (maximum) across all x.

3) Jensen–Shannon (JS) Divergence: The Jensen–Shannon divergence is a symmetric and smoothed version of Kullback–Leibler divergence, used to compare two probability distributions P and Q (e.g., feature histograms). It is defined as:

$$JSD(P \parallel Q) = \tfrac{1}{2} D_{KL}(P \parallel M) + \tfrac{1}{2} D_{KL}(Q \parallel M) \tag{7}$$

where $M = \frac{1}{2}(P+Q)$ is the average distribution, and D_{KL} is the Kullback–Leibler divergence. In this study, we report the Jensen–Shannon distance (JS), which is the square root of the JS divergence after base-2 normalisation. JS Distance is symmetric and bounded in [0, 1] (0 = identical, 1 = maximally different), which makes it easier to interpret. [21].

D. Model Training and Evaluation

Finally, to link statistical drift to practical consequences, non-incremental classifiers were trained to predict obfuscation categories using a 70/30 split of each quarterly bin. The models were trained in the training set for a given quarter and then evaluated on the test data held from the same quarter and all subsequent quarters. For classification, three machine learning models were employed: a linear *Passive Aggressive Classifier*, *Multinomial Naive Bayes*, and a tree-based *Gradient Boosting Classifier*. This setup simulates a static deployment scenario and allows one to quantify how performance degrades when models are not adapted to evolving data distributions. The performance of the models was evaluated using various metrics, including accuracy, precision, recall, and F1-score. Due to the class imbalance of the dataset, a weighted F1-score was used, as it accounts for the imbalance by averaging the F1 of each class according to its support. This was also the primary metric for correlation analysis between drift and classifier performance.

$$\text{F1}_{\text{weighted}} = \sum_{c \in C} \frac{n_c}{N} \cdot \frac{2 \times \text{Precision}_c \times \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}, \tag{8}$$

where n_c is the number of samples in class c, N is the total number of samples, and C is the set of classes.

E. Statistical Analysis

To examine the relationship between feature drift and model performance, Spearman's rank correlation was used. Spearman correlation is a non-parametric measure that evaluates the strength and direction of the monotonic association between two variables [23]. In this study, it was applied to assess the degree of correlation between drift metrics (PSI, JS, KS) and the weighted F1-scores of the classifiers. Given a sample size n and rank differences d_i between paired observations, the Spearman correlation coefficient ρ is defined as:

$$\rho = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)}. \tag{9}$$

A negative correlation indicates that higher drift values are associated with lower classification performance, thereby directly linking feature instability to model degradation.

F. Implementation Details

The code for the audio transformation, feature extraction, drift analysis, and classification pipelines was conducted in Python. For audio processing and feature extraction, the soundfile and Librosa (v0.9.2) libraries were used. The drift tests were implemented using NumPy, pandas, and scipy.stats (ks_2samp for the Kolmogorov–Smirnov test). Spearman rank correlations were computed using scipy.stats.spearmanr. Additional preprocessing utilities made use of re and pandas. Graphs were generated using Matplotlib and Seaborn.

Model training and evaluation used scikit-learn, which was also used to compute performance metrics such as accuracy and weighted F1-score. All development and analysis were performed in Jupyter Notebook on a Dell PC running Windows 11 Pro (version 24H2), equipped with a 12th Gen Intel(R) Core(TM) i9-12900 CPU @ 2.40 GHz, 32 GB of RAM, and a 64-bit operating system.

IV. Results

This section presents the experimental results evaluating the robustness of audio features under concept drift in Android malware detection. Drift was assessed using three statistical measures including Population Stability Index (PSI), Jensen–Shannon Distance (JS), and Kolmogorov–Smirnov statistic (KS), across consecutive temporal bins. The study also analysed family-level drift rankings, stability bucket distributions, and drift–time gap relationships. Finally, Spearman correlations were computed between drift scores and non-incremental classifier performance (F1-weighted), using both linear (Passive Aggressive) and tree-based (Gradient Boosting) models.

A. Per-feature drift tests (PSI, JSD, KS)

Table II presents the aggregated drift statistics. The most unstable families were ZCR and Chroma_STFT, with median PSI values above 0.45 and drift detected in over 60% of transitions. Spectral Centroid and RMS also showed high susceptibility to drift, with median PSI values exceeding 0.32. These findings were further supported by KS test results.

Core audio features such as MFCCs and Mel-Spectrograms displayed intermediate stability (median PSI 0.22–0.23; drift prevalence 45–48%), indicating that they were neither the most drift-prone nor the most resilient.

Features including Spectral Flatness, Spectral Bandwidth, and Spectral Rolloff remained largely stable, as they recorded very low median PSI values (< 0.14) and drift prevalence below 20%. Across all features, Jensen–Shannon Distance (JS) values for were moderate (≈ 0.27–0.29 for the most unstable families), while stable families remained lower ($\approx 0.12 - 0.14$), indicating that while the distributions diverged across consecutive bins, their overall shapes were partially preserved. Kolmogorov-Smirnov (KS) statistics were consistent with the PSI rankings, with unstable families recording higher medians (> 0.22) than stable families (~ 0.11–0.13). Overall, PSI, JS, and KS converged on the same stable and drift-prone feature families, reinforcing the robustness of these results.

TABLE II
Ranking of Feature Families by Drift Severity Across Statistical Metrics

FeatureFamily	mean_PSI	median_PSI	mean_JS	median_JS	mean_KS	median_KS	pct_moderate	pct_drift
ZCR	0.473	0.517	0.260	0.292	0.235	0.265	18.18	72.73
Chroma_STFT	0.527	0.467	0.246	0.284	0.238	0.271	21.97	63.64
Spectral_Centroid	0.459	0.419	0.255	0.270	0.220	0.229	9.09	63.64
RMS	0.299	0.326	0.213	0.240	0.148	0.154	18.18	63.64
Poly_Features	0.287	0.259	0.205	0.214	0.175	0.157	22.73	50.00
Chroma_CQT	0.280	0.250	0.201	0.210	0.183	0.182	20.45	50.00
Mel_Spectrogram	0.304	0.234	0.205	0.202	0.174	0.149	24.72	48.37
MFCC	0.280	0.218	0.200	0.195	0.168	0.151	27.95	45.45
Chroma_CENS	0.269	0.191	0.188	0.183	0.167	0.153	30.30	38.64
Spectral_Contrast	0.210	0.142	0.173	0.159	0.135	0.119	29.87	32.47
Spectral_Rolloff	0.168	0.139	0.159	0.157	0.128	0.125	45.45	18.18
Spectral_Bandwidth	0.169	0.109	0.154	0.139	0.115	0.107	54.55	9.09
Spectral_Flatness	0.133	0.091	0.138	0.126	0.114	0.135	36.36	9.09

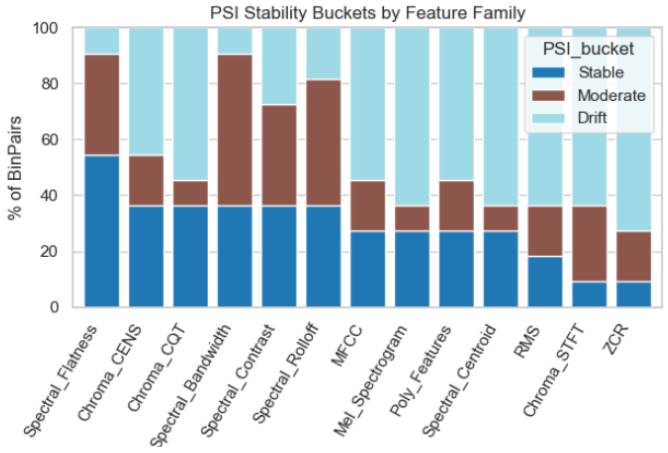


Fig. 3. Stability_bucket

Figure 3 illustrates the distribution of stability outcomes (Stable, Moderate, Drift) across feature families. Consistent with Table I, ZCR and Chroma_STFT show the highest instability, drifting in more than 60% of temporal transitions. Spectral Centroid and RMS also exhibit substantial drift proportions, whereas Spectral Flatness and Spectral Bandwidth remain largely stable, exceeding 80% stability across bins.

Figure 4 tracks median PSI values over time for selected families. The results highlight clear divergence between drift-prone and stable features: ZCR and Chroma_STFT frequently surpass the drift threshold (0.25), peaking above 0.8 in certain quarters, while Spectral Flatness and Spectral Bandwidth remain consistently below the moderate-change threshold (0.1). This temporal view reinforces the conclusion that drift affects feature families unevenly, with some descriptors remaining resilient, while others degrade rapidly.

B. Drift vs Time Gap

Figures 5, 6, and 7 plot drift values against the temporal gap between bins for two drift-prone families (ZCR, Chroma_STFT) and one stable family (Spectral Flatness). The results show a clear accumulation effect across all three metrics (PSI, JS, KS). In PSI, ZCR, and Chroma STFT cross the 0.10 band early (gaps $\approx$ 2–3) and exceed 0.25 by gaps $\approx 3 - 4$, approaching the upper display cap at larger gaps, whereas Spectral Flatness remains below 0.25 until long gaps. For JSD and KS, the horizontal lines are reference guides

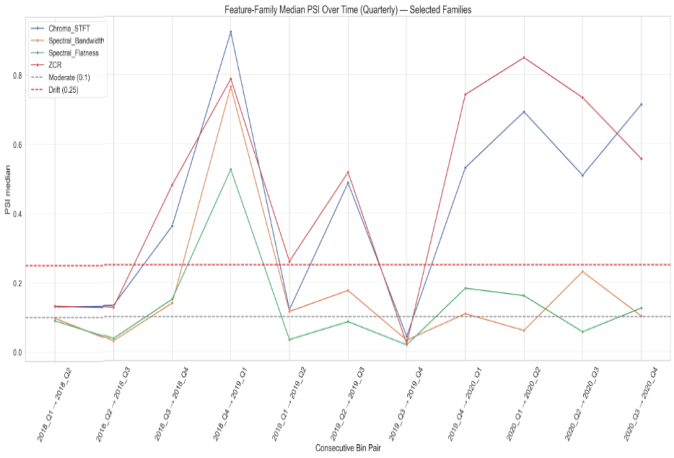


Fig. 4. Temporal PSI feature median

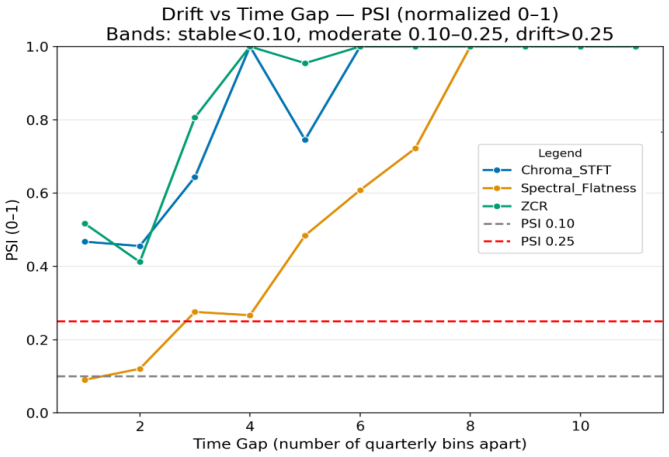


Fig. 5. Drift vs Time Gap (PSI)

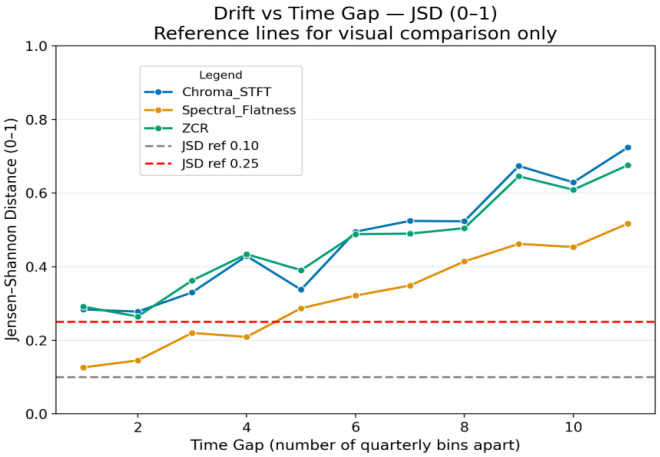


Fig. 6. Drift vs Time Gap (JS)

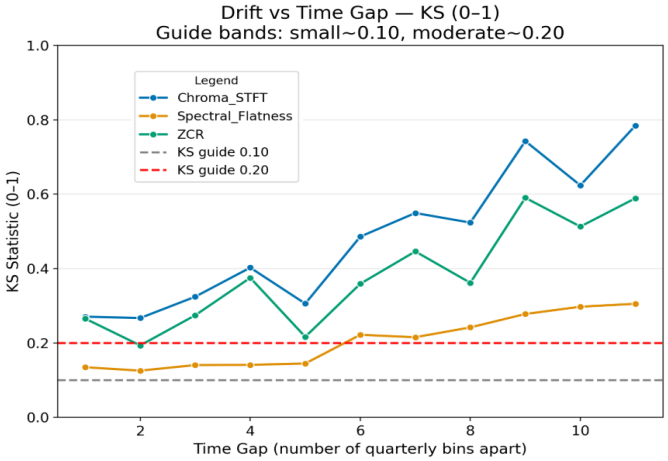


Fig. 7. Drift vs Time Gap (KS)

(not thresholds); the same upward trend is observed, with peaks typically around 0.6–0.7 (JS) and 0.5–0.8 (KS) for drift-prone families, while Spectral Flatness remains lower ($\approx$ 0.2–0.35).These findings complement Figures 3 and 4 by confirming that not only do some families drift more frequently, but their instability also intensifies with increasing temporal separation, underscoring the cumulative nature of concept drift.

C. Feature Stability Rankings

Figures 8, 9 and 10 rank the feature families by their median drift scores, providing a direct comparison of relative stability. The ordering is consistent with earlier findings: Spectral Flatness, Spectral Bandwidth, and Spectral Rolloff emerge as the most stable descriptors, while ZCR and Chroma_STFT are the least stable. Intermediate families such as MFCC, Chroma CQT, and Mel Spectrogram fall in the mid-range, showing partial resilience but still prone to moderate drift. By consolidating PSI, JS, and KS into a composite view, this ranking reinforces the divergence between highly drift-prone temporal features and more stable spectral descriptors.

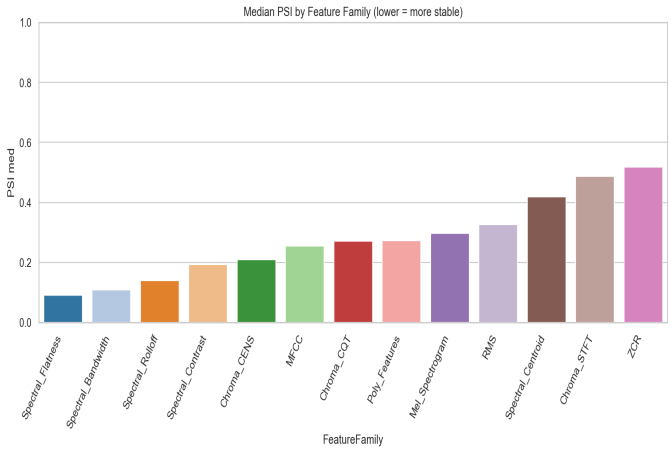


Fig. 8. Feature Family Median PSI

D. Correlation with Model Performance (Spearman)

Table III presents the Spearman correlations between drift metrics and non-incremental F1-weighted performance. All models exhibited strong negative relationships, confirming that

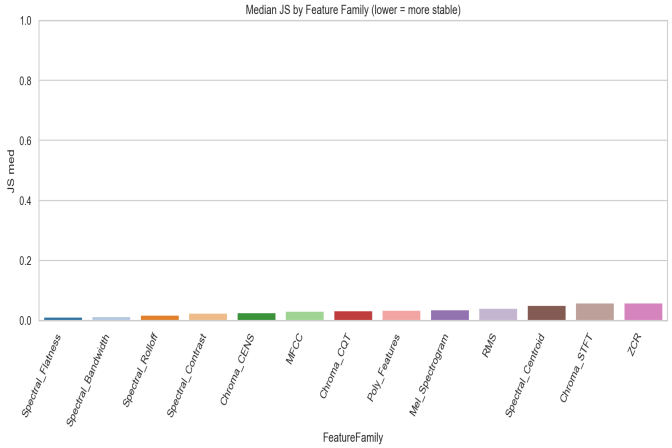


Fig. 9. Feature Family Median JS

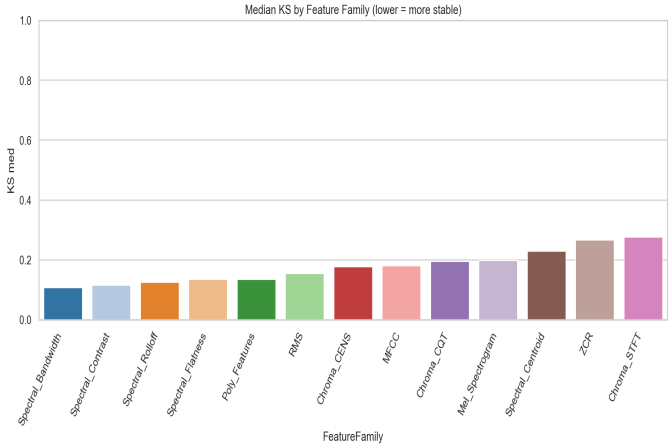


Fig. 10. Feature Family Median KS

TABLE III
MEDIAN SPEARMAN CORRELATIONS (ρ) BETWEEN DRIFT METRICS (PSI, JS, KS) AND NON-INCREMENTAL F1-WEIGHTED PERFORMANCE (QUARTERLY). NEGATIVE VALUES INDICATE THAT HIGHER DRIFT COINCIDES WITH LOWER PERFORMANCE.

Model	PSI (ρ, range)	JS (ρ, range)	KS (ρ, range)
PassiveAggressive (linear)	–0.46 (–0.53 … –0.32)	–0.48 (–0.54 … –0.33)	–0.47 (–0.53 … –0.21)
Gradient Boosting (tree)	–0.64 (–0.66 … –0.49)	–0.64 (–0.66 … –0.47)	–0.62 (–0.68 … –0.29)
Multinomial Naïve Bayes	–0.81 (–0.88 … –0.54)	–0.84 (–0.88 … –0.54)	–0.81 (–0.89 … –0.16)

classifier performance decreases as feature drift intensifies. The Passive Aggressive (linear) classifier exhibited moderate correlations ($\rho \approx -0.46$ to -0.47), indicating measurable yet limited sensitivity to evolving distributions. The Gradient Boosting classifier demonstrates stronger effects ($\rho \approx -0.62$ to -0.64), revealing that ensemble models degrade more sharply under drift. The Multinomial Naïve Bayes model displayed the strongest negative associations ($\rho \approx -0.81$ to -0.84), highlighting the model's high dependence on stable feature distributions and vulnerability to temporal data shifts.

Figure 11 provides a grouped bar chart showing the feature-family correlations for the Gradient Boosting model. All correlations are negative, confirming that drift consistently

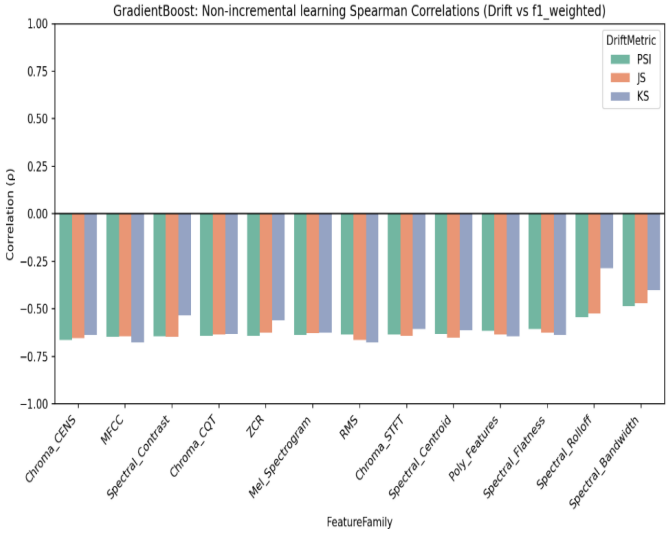


Fig. 11. Spearman Correlation (Drift vs F1_weighted)

coincides with lower F1 performance. Temporal and spectral features such as MFCC, Chroma variants, and the Mel-spectrogram displayed the strongest associations ($\rho \approx -0.63$ to -0.68), highlighting their sensitivity to evolving distributions. In contrast, the features Spectral Rolloff and Spectral Bandwidth exhibited weaker effects (around -0.29 to -0.40). Overall, the plot reinforces that features most prone to drift contribute most to model degradation under non-incremental learning.

V. DISCUSSION

The results provide evidence that audio features are not stable over time. Drift experiments showed that families such as ZCR, Chroma STFT, and Spectral Centroid are highly prone to distribution shifts, while features such as Spectral Rolloff and Spectral Bandwidth remained comparatively stable. The stacked prevalence plots revealed that some feature families drift in more than 60% of temporal transitions, and the drift versus gap curves confirmed that the drift or instability intensifies as the time interval between bins increases.

These findings have important implications for malware detection research. First, they highlight that using audio features 'as is' without any adaptation methods will result in the degradation of model performance over time. Second, not all features contribute equally to drift; some descriptors are inherently more robust, which suggests opportunities for feature selection or weighting. Finally, the negative Spearman correlations between drift scores and non-incremental model performance confirm that drift is not just a statistical artifact but has a practical impact on classifier accuracy and F1 performance.

VI. CONCLUSION

This study investigated the robustness of audio features to concept drift in Android malware detection. Using PSI, JS, and KS, the study showed that drift is pervasive and uneven across

feature families. Audio features, ZCR, and Chroma features were found to be highly unstable, while spectral features like Spectral Flatness and bandwidth were relatively stable. Negative Spearman correlations with model performance also confirm that drift degrades classifier effectiveness under non-incremental or static setups. These findings establish audio features as vulnerable to concept drift and provide a baseline for future drift-aware approaches.

VII. Future Work

While this study established that audio features suffer from concept drift in Android malware detection, future work should address mitigation. Areas for future research include incremental learning, adaptive learning (incremental or online updates), drift-triggered retraining, and feature-level selection that prioritises stable families (e.g., Spectral Flatness, Bandwidth) over drift-prone ones (e.g., ZCR, Chroma). As part of future a valuable extension would be to compare the robustness of audio features against traditional Android features such as API calls and permissions, together with exploring multimodal combinations (audio + API + images) for long-term resilience.

References

[1] Kaspersky, "Android vs. iOS Security Comparison," Jul. 2020, section: Threats. [Online]. Available: https://www.kaspersky.com/resource-center/threats/android-vs-iphone-mobile-security

[2] W. Cho, H. Lee, S. Han, Y. Hwang, and S. -j. Cho, "Sustainability of Machine Learning-based Android Malware Detection Using API calls and Permissions," in *2022 IEEE Fifth International Conference on Artificial Intelligence and Knowledge Engineering (AIKE)*, Sep. 2022, pp. 18–25, journal Abbreviation: 2022 IEEE Fifth International Conference on Artificial Intelligence and Knowledge Engineering (AIKE).

[3] F. M. Darus, N. A. A. Salleh, and A. F. Mohd Ariffin, "Android Malware Detection Using Machine Learning on Image Patterns," in *2018 Cyber Resilience Conference (CRC)*, Nov. 2018, pp. 1–2. [Online]. Available: https://ieeexplore.ieee.org/document/8626828

[4] P. Tarwireyi, A. Terzoli, and M. O. Adigun, "Using multi-audio feature fusion for android malware detection," *Computers and Security*, vol. 131, no. C, Aug. 2023, number: C. [Online]. Available: https://doi.org/10.1016/j.cose.2023.103282

[5] M. Farrokhmanesh and A. Hamzeh, "A novel method for malware detection using audio signal processing techniques," in *2016 Artificial Intelligence and Robotics (IRANOPEN)*, Apr. 2016, pp. 85–91. [Online]. Available: https://ieeexplore.ieee.org/document/7529495

[6] D. Hu, Z. Ma, X. Zhang, P. Li, D. Ye, and B. Ling, "The concept drift problem in android malware detection and its solution," *Security and Communication Networks*, vol. 2017, 2017, publisher: Hindawi Limited. [Online]. Available: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85030769063&doi=10.1155%2f2017%2f4956386&partnerID=40&md5=17fc98c50a92199b95985ec14911650e

[7] Alon Gubkin, "Concept Drift in Machine Learning 101," Apr. 2021. [Online]. Available: https://coralogix.com/ai-blog/concept-drift-in-machine-learning-101/

[8] G. Hovakimyan and J. M. Bravo, "Evolving Strategies in Machine Learning: A Systematic Review of Concept Drift Detection," *Information*, vol. 15, no. 12, p. 786, Dec. 2024, publisher: Multidisciplinary Digital Publishing Institute. [Online]. Available: https://www.mdpi.com/2078-2489/15/12/786

[9] T. M. T. Pham, K. Premkumar, M. Naili, and J. Yang, "Time to Retrain? Detecting Concept Drifts in Machine Learning Systems," Aug. 2025, arXiv:2410.09190 [cs]. [Online]. Available: http://arxiv.org/abs/2410.09190

[10] F. Hinder, V. Vaquet, J. Brinkrolf, and B. Hammer, "Model-based explanations of concept drift," *Neurocomputing*, vol. 555, p. 126640, Oct. 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0925231223007634

[11] F. Ceschin, M. Botacin, H. Gomes, F. Pinagé, L. Oliveira, and A. Grégio, "Fast & Furious: On the modelling of malware detection as an evolving data stream[Formula presented]," *Expert Systems with Applications*, vol. 212, 2023, publisher: Elsevier Ltd. [Online]. Available: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85137569528&doi=10.1016%2fj.eswa.2022.118590&partnerID=40&md5=ce4e6b4ec8ffdda606b8123971539ce5

[12] C. Gao, G. Huang, H. Li, B. Wu, Y. Wu, and W. Yuan, "A Comprehensive Study of Learning-Based Android Malware Detectors Under Challenging Environments," in *2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE)*, Apr. 2024, pp. 115–127, journal Abbreviation: 2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE).

[13] F. Barbero, F. Pendlebury, F. Pierazzi, and L. Cavallaro, "Transcending TRANSCEND: Revisiting Malware Classification in the Presence of Concept Drift," in *Proc. IEEE Symp. Secur. Privacy*, vol. 2022-May. Institute of Electrical and Electronics Engineers Inc., 2022, pp. 805–823, journal Abbreviation: Proc. IEEE Symp. Secur. Privacy. [Online]. Available: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85135967664&doi=10.1109%2fSP46214.2022.9833659&partnerID=40&md5=0bf2f30d2da878b21f6c98818aa25ea0

[14] A. Guerra-Manzanares, H. Bahsi, and M. Luckner, "Leveraging the first line of defense: a study on the evolution and usage of android security permissions for enhanced android malware detection," *Journal of Computer Virology and Hacking Techniques*, vol. 19, no. 1, pp. 65–96, 2023, publisher: Springer-Verlag Italia s.r.l. [Online]. Available: https://www.scopus.com/inward/record.uri?eid=2-s2.0-85137067746&doi=10.1007%2fs11416-022-00432-3&partnerID=40&md5=c6a057ba6d71ce9c677291d35b5d0708

[15] P. Tarwireyi, A. Terzoli, and M. O. Adigun, "Meta-SonifiedDroid: Metaheuristics for Optimizing Sonified Android Malware Detection," *IEEE Access*, pp. 1–1, 2024, conference Name: IEEE Access. [Online]. Available: https://ieeexplore.ieee.org/document/10559579/?arnumber=10559579

[16] O. E. Kural, E. Kiliç, and C. Aksaç, "Apk2Audio4AndMal: Audio Based Malware Family Detection Framework," *IEEE Access*, vol. 11, pp. 27 527–27 535, 2023. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/10073518

[17] S. Kumar, D. Mishra, B. Panda, and S. K. Shukla, "AndroOBFS: Time-tagged Obfuscated Android Malware Dataset with Family Information," in *2022 IEEE/ACM 19th International Conference on Mining Software Repositories (MSR)*, May 2022, pp. 454–458, iSSN: 2574-3864. [Online]. Available: https://ieeexplore.ieee.org/document/9796156

[18] T. Giannakopoulos and A. Pikrakis, "Chapter 4 - Audio Features," in *Introduction to Audio Analysis*, T. Giannakopoulos and A. Pikrakis, Eds. Oxford: Academic Press, Jan. 2014, pp. 59–103. [Online]. Available: https://www.sciencedirect.com/science/article/pii/B9780080993881000042

[19] P. Raghuwanshi and R. Kaushik, "Insect Classification Using Mel-CSTFT: A Fusion of Mel Spectrogram and Chroma STFT Features," in *2024 First International Conference on Electronics, Communication and Signal Processing (ICECSP)*, Aug. 2024, pp. 1–5. [Online]. Available: https://ieeexplore.ieee.org/document/10698728

[20] L. Nataraj, T. M. Mohammed, T. Nanjundaswamy, S. Chikkagoudar, S. Chandrasekaran, and B. S. Manjunath, "OMD: Orthogonal Malware Detection Using Audio, Image, and Static Features," Nov. 2021, arXiv:2111.04710 [cs]. [Online]. Available: http://arxiv.org/abs/2111.04710

[21] N. Harshit and K. Mounvik, "Improving Real-Time Concept Drift Detection using a Hybrid Transformer-Autoencoder Framework," Aug. 2025, arXiv:2508.07085 [cs]. [Online]. Available: http://arxiv.org/abs/2508.07085

[22] F. Bayram, B. S. Ahmed, and A. Kassler, "From concept drift to model degradation: An overview on performance-aware drift detectors," *Knowledge-Based Systems*, vol. 245, p. 108632, Jun. 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0950705122002854

[23] M. Ashawa and S. Morris, "Modeling Correlation between Android Permissions Based on Threat and Protection Level Using Exploratory Factor Plane Analysis," *Journal of Cybersecurity and Privacy*, vol. 1, no. 4, pp. 704–742, Dec. 2021, publisher: Multidisciplinary Digital Publishing Institute. [Online]. Available: https://www.mdpi.com/2624-800X/1/4/35